

INTRODUCTION OF C++

1. C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming.
2. C++ is a middle-level language, as it encapsulates both high and low level language features.
3. C++ supports the object-oriented programming,
4. C++ Deals Real world Entity.

History of C++

C++ programming language was developed in 1980 by Bjarne Stroustrup at bell laboratories of AT&T (American Telephone & Telegraph), located in U.S.A.

Bjarne Stroustrup is known as the founder of C++ language.

It was develop for adding a feature of OOP (Object Oriented Programming) in C++ without significantly changing the C component.

C++ programming is "relative" (called a superset) of C, it means any valid C program is also a valid C++ program.

Features of C++

1. Simple
2. Abstract Data types
3. Machine dependent or Portable
4. Mid-level programming language
5. Structured programming language
6. Rich Library
7. Memory Management
8. Quicker Compilation
9. Pointers
10. Recursion
11. Extensible
12. Object-Oriented
13. Compiler based
14. Reusability
15. Strongly typed language
16. Redefine Existing Operators
17. Modelling Real-World Problems.

Difference Between C and C++

Parameter	C	C++
Programming Paradigm	C is a structural or procedural programming language.	C is a structural as well as an object-oriented programming language.
History	C was developed by scientist Dennis Ritchie in 1972 at Bell Laboratories.	C was developed by Bjarne Stroustup in 1979.
Approach	C follows a top-down approach	C follows the bottom-up approach.
Keywords	C contains 32 keywords	C++ contains 63 keywords.
Data Types	C supports built-in data types.	C++ support both built-in and user-defined data types.
File Extension	.c is the file extension for C programming language	.cpp is the file extension for C++ programming language
Header File	<stdio.h> header file is used by C language	<iostream.h> header file is used by C++ language
Allocation and Deallocation of Memory	In C language, we use calloc() and malloc() for dynamic allocation of memory and free() for deallocation of memory.	In C++ language, we use a new operator for the allocation of memory and a delete operator for the deallocation of memory.
Access Modifier	C language does not support access modifier	C++ support access modifier
Security	C does not have any security features so it can be manipulated by outsider	C++ is a secure language as it offers security features such as data hiding and encapsulation
Reference Variable	C does not support reference variable	C++ support reference variable
Function Overloading and Function Overriding	C don't supports function overloading and function overriding	C++ supports function overloading and function overriding
Exception Handling	C does not support exception handling directly, it uses the function that support exception handling	C++ directly support exception handling with the help of try – catch block

Program Division	C is a procedural language, so code written in C are divided in separate blocks known as function	C++ is a object oriented language, so code and divided into classes and objects
Inline Function	C doesn't support inline function	C++ support inline function
Driven Type	C is known as function driven language	C is known as object driven language
Compatibility	Code written in C language can be run on C++ compiler as C is the foundational language	Code written in C++ language can be run on C compiler as C++ language includes OOP's concept
Data and Function	In C, the data and function are separated as it is a procedural programming language	In C++, the data and function are encapsulated as it is a object oriented programming language
Input and Output Function	In C scanf() and printf() functions are used to take the input and output respectively	In C++ cin and cout functions are used to take the input and output respectively
Application Development	C language is more suitable for low level implementation such as network driver, text editor, assembler, etc	C++ language is more suitable for high level implementation such as game development, smartwatches, embedded systems, etc
Namespace	To prevent the collision and organize the code, namespace is needed but C does not support that	C++ support the namespace
Used By	MySQL, Windows Kerne, Oracle Database, Telegram messenger, etc	Google Chrome, Torque 3-D game, Microsoft Office, etc

Similarity Between C and C++

- Both C and C++ follows similar code structure
- They both have the similar compilation
- They also follow similar coding syntax
- C++ have extended grammar but the basic grammar is same as C
- Their memory model is very close to the hardware

- The both languages have the similar notation of stack, heap and file scope variable.
- Mostly all C operators and keywords are present in C++ as well.

Procedural Oriented programming Language

A procedural language is a type of computer programming language that specifies a series of well-structured steps and procedures within its programming context to compose a program. It contains a systematic order of statements, functions and commands to complete a computational task or program.

The characteristics of procedural programming are:

1. Procedural programming follows a top-down approach.
2. The program is divided into blocks of codes called functions, where each function performs a specific task.
3. Procedural programs model real-world processes as 'procedures' operating on 'data'.
4. The data and functions are detached from each other.
5. The data moves freely in a program.
6. It is easy to follow the logic of a program.
7. A function can access other function's data by calling that function.

Object oriented Programming Language

Object-oriented programming has a sweeping impact because it appeals at multiple levels and promises faster and cheaper development and maintenance. It follows a bottom-up approach to develop applications.

The word **object-oriented** is the combination of two words i.e. **object** and **oriented**. The dictionary meaning of the object is an article or entity that exists in the real world. The meaning of oriented is interested in a particular kind of thing or entity. In layman's terms, it is a programming pattern that rounds around an object or entity are called **object-oriented programming**.

Key Features of oops

- Emphasis on data rather than procedure
- Programs are divided into entities known as objects
- Data Structures are designed such that they characterize objects
- Functions that operate on data of an object are tied together in data structures
- Data is hidden and cannot be accessed by external functions
- Objects communicate with each other through functions
- New data and functions can be easily added whenever necessary
- Follows bottom up design in program design.

Basic Concept of Object oriented programming language

1. Data Abstraction
2. Encapsulation
3. inheritance
4. polymorphism
5. class
6. object

Class:

- a. Class is user define data type
- b. Class is collection of object which act as a building blocks.
- c. Class is logical Entity.
- d. Class can be defined as blue print of object.
- e. A class contains data members (variables) and member functions.

Object:

- a. An Object can be defined as an entity that has a state and behaviour, or in other words, anything that exists physically in the world is called an object.
- b. Object is instance of class.
- c. Every object belongs to specific class.

Inheritance:

- a. When one object acquires all the properties and behaviours of parent object i.e. known as inheritance.
 - b. It provides code reusability.
 - c. It is used to achieve runtime polymorphism.
 - d. It follows the parent and Child relationship, where parent class is known as super class or base class and child class is known as sub class or derived class.
-
1. Sub class - Subclass or Derived Class refers to a class that receives properties from another class.
 2. Super class - The term "Base Class" or "Super Class" refers to the class from which a subclass inherits its properties.

Types of Inheritance

1. Single level Inheritance

2. Multilevel Inheritance
3. Multiple Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Encapsulation:

1. Encapsulation is the concept of wrapping together of data and information in a single unit.
2. It is most striking features of class the data is not accessible to outer world and only the main functionality can be accessible to outer world.

Polymorphism:

1. Polymorphism means many forms.
2. It is the ability to take more than one form.
3. It is a feature that provides a function or an operator with more than one definition.
4. It can be implemented using function overloading, operator overload, function overriding, and virtual functions.

Types of Polymorphism:

1. Compile time Polymorphism
2. Run time Polymorphism

Dynamic Binding:

In dynamic binding, a decision is made at runtime regarding the code that will be run in response to a function call. For this, C++ supports virtual functions.

Message Passing:

All communication between objects is done via message is called message passing, as people exchange information similarly the sending and receiving of information by the object is said to be message passing, to make possible message passing the following things have to be followed to be done :

- We have to create classes that define objects and its behaviour.
- Then Creating the objects from class definitions
- Calling and connecting the communication among objects

Structure of C++ Program

The C++ program is written using a specific template structure. The structure of the program written in C++ language is as follows:

Documentation
Link Section
Definition Section
Global Declaration Section
Function definition Section
Main Function

Skeleton of C Program

C++ Basic Input/Output

C++ I/O operation is using the stream concept. Stream is the sequence of bytes or flow of data. It makes the performance fast.

If bytes flow from main memory to device like printer, display screen, or a network connection, etc, this is called as **output operation**.

If bytes flow from device like printer, display screen, or a network connection, etc to main memory, this is called as **input operation**.

I/O Library Header Files

Let us see the common header files used in C++ programming are:

Header File	Function and Description
<iostream>	It is used to define the cout , cin and cerr objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.
<iomanip>	It is used to declare services useful for performing formatted I/O, such as setprecision and setw .
<fstream>	It is used to declare services for user-controlled file processing.

Standard output stream (cout):

The **cout** is a predefined object of **ostream** class. It is connected with the standard output device, which is usually a display screen. The cout is used in conjunction with stream insertion operator (<<) to display the output on a console

Standard input stream (cin)

The **cin** is a predefined object of **istream** class. It is connected with the standard input device, which is usually a keyboard. The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.

Standard end line (endl)

The **endl** is a predefined object of **ostream** class. It is used to insert a new line characters and flushes the stream.

Concept of Name space

A **namespace** is designed to overcome this difficulty and is used as additional information to differentiate similar functions, classes, variables etc. with the same name available in different libraries. Using namespace, you can define the context in which names are defined. In essence, a namespace defines a scope.

Defining a Namespace

A namespace definition begins with the keyword **namespace** followed by the namespace name as follows –

```
namespace namespace_name {  
    // code declarations  
}
```

C++ Tokens

A C++ program is composed of tokens which are the smallest individual unit. Tokens can be one of several things, including keywords, identifiers, constants, operators, or punctuation marks.

There are 6 types of tokens in c++:

1. Keyword
2. Identifiers
3. Constants
4. variable
5. Special symbols
6. Operators

1. Keywords:

In C++, keywords are reserved words that have a specific meaning in the language and cannot be used as identifiers. Keywords are an essential part of the C++ language and are used to perform specific tasks or operations.

There are 95 keywords in c++:

alignas (since C++11)	alignof (since C++11)	And
And	and_eq	Asm
atomic_cancel (TM TS)	atomic_commit (TM TS)	atomic_noexcept (TM TS)
auto(1)	Bitand	Bitor
Bool	Break	Case
Catch	Char	char16_t (since C++11)
char32_t (since C++11)	class(1)	Compl
concept (since C++20)	Const	constexpr (since C++11)
const_cast	Continue	co_await (coroutines TS)
co_return (coroutines TS)	co_yield (coroutines TS)	decltype (since C++11)
default(1)	delete(1)	Do
Double	dynamic_cast	Else
Enum	Explicit	export(1)
extern(1)	False	Float
For	Friend	Goto
If	import (modules TS)	inline(1)
Int	Long	module (modules TS)
mutable(1)	Namespace	New
noexcept (since C++11)	Not	not_eq
nullptr (since C++11)	Operator	Or
or_eq	Private	Protected
Public	register(2)	reinterpret_cast
requires (since C++20)	Return	Short
Signed	sizeof(1)	Static
static_assert (since C++11)	static_cast	struct(1)
Switch	synchronized (TM TS)	Template
This	thread_local (since C++11)	Throw
True	Try	Typedef
Typeid	Typename	Union
Unsigned	using(1)	Virtual
Void	Volatile	wchar_t
While	Xor	xor_eq

2. Identifiers

In C++, an identifier is a name given to a variable, function, or another object in the code. Identifiers are used to refer to these entities in the program, and they can

consist of letters, digits, and underscores. However, some rules must be followed when choosing an identifier:

- The first character must be a letter or an underscore.
- Identifiers cannot be the same as a keyword.
- Identifiers cannot contain any spaces or special characters except for the underscore.
- Identifiers are case-sensitive, meaning that a variable named "myVariable" is different from a variable named "myvariable".

Here are some examples of valid and invalid identifiers in C++:

- **Valid:**
 - my_variable
 - student_name
 - balance_due
- **Invalid:**
 - my variable (contains a space)
 - student# (contains a special character)
 - int (same as a keyword)

Operator in c++

All c operator are used in c++

1. << *Insertion*
2. >> *Exertion*
3. :: *Scope resolution* operator
4. Endl- line feed operator
5. New: allocate new memory
6. Delete: DE allocate memory
7. ->pointer to member operator
8. . pointer to member operator
9. Setw: field width operator
10. setPrecision();

Control Statement in C++

Same as C language

What is the structure in C++?

A structure is a grouping of variables of various data types referenced by the same name. A structure declaration serves as a template for creating an instance of the structure.

Syntax:

The syntax of the structure is as follows

1. Struct Structurename
2. {
3. Struct_member1;
4. Struct_member2;
5. Struct_member3;
6. .
7. .
8. .
9. Struct_memberN;
10. };

The "struct" keyword indicates to the compiler that a structure has been declared. The "structurename" defines the name of the structure. Since the structure declaration is treated as a statement, so it is often ended by a semicolon.

Reference Variable

Reference variable is an alternate name of already existing variable. In other words, we can say that reference variable is the alternative name to your variable. The operator '&' is used to declare reference variable.

The following is the syntax of reference variable.

`datatype variable_name; // variable declaration`

`datatype &refer_var = variable_name; // reference variable`

Here,

datatype – The datatype of variable like int, char, float etc.

variable_name – This is the name of variable given by user.

refer_var – The name of reference variable.

Reference vs pointer

1. Once reference is created, it cannot be later made reference of another object, it cannot be reseted .this is often done with pointer .
2. Reference cannot be NULL. Pointer are often made NULL to indicate that they are not pointing to any valid thing.
3. A reference must be initialize when declared. There are no such restrictions with pointer.

What is Class in C++?

A class in C++ is similar to a C structure in that it consists of a list of data members and a set of operations performed on the class. In other words, a class is the building block of Object-Oriented programming. It is a user-defined object type with its own set of data members and member functions that can be accessed

and used by creating a class instance. A C++ class is similar to an object's blueprint.

Syntax:

The structure and the class are syntactically similar. The syntax of class in C++ is as follows:

1. `class class_name`
2. `{`
3. `// private data members and member functions.`
4. `Access specifier;`
5. `Data member;`
6. `Member functions (member list){ . . }`
7. `};`

In this syntax, the `class` is a keyword to indicate the compiler that a class has been declared. OOP's main function is data hiding, which is achieved by having three access specifiers: "public", "private", and "safe". If no access specifier is specified in the class when declaring data members or member functions, they are all considered private by default.

Sturcture vs Class

Features	Structure	Class
Definition	A structure is a grouping of variables of various data types referenced by the same name.	In C++, a class is defined as a collection of related variables and functions contained within a single structure.
Basic	If no access specifier is specified, all members are set to 'public'.	If no access specifier is defined, all members are set to 'private'.
Declaration	<pre>struct structure_name{ type struct_member 1; type struct_member 2; type struct_member 3; . type struct_memberN; };</pre>	<pre>class class_name{ data member; member function; };</pre>
Instance	Structure instance is called the 'structure variable'.	A class instance is called 'object'.

Inheritance	It does not support inheritance.	It supports inheritance.
Memory Allocated	Memory is allocated on the stack.	Memory is allocated on the heap.
Nature	Value Type	Reference Type
Purpose	Grouping of data	Data abstraction and further inheritance.
Usage	It is used for smaller amounts of data.	It is used for a huge amount of data.
Null values	Not possible	It may have null values.
Requires constructor and destructor	It may have only parameterized constructor.	It may have all the types of constructors and destructors.

Example of classes and Object

Array of object

An [array](#) in [C/C++](#) or be it in any programming language is a collection of similar data items stored at contiguous memory locations and elements can be accessed randomly using indices of an array. They can be used to store the collection of primitive data types such as int, float, double, char, etc of any particular type. To add to it, an array in C/C++ can store derived data types such as structures, pointers, etc. Given below is the picture representation of an array.

When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

```
ClassName ObjectName[number of objects];
```

Scope Resolution Operator

The scope resolution operator is used to reference the global variable or member function that is out of scope. Therefore, we use the scope resolution operator to access the hidden variable or function of a program. The operator is represented as the double colon (::) symbol.

For example, when the global and local variable or function has the same name in a program, and when we call the variable, by default it only accesses the inner or

local variable without calling the global variable. In this way, it hides the global variable or function. To overcome this situation, we use the scope resolution operator to fetch a program's hidden variable or function.

Uses of the scope resolution Operator

1. It is used to access the hidden variables or member functions of a program.
2. It defines the member function outside of the class using the scope resolution.
3. It is used to access the static variable and static function of a class.
4. The scope resolution operator is used to override function in the Inheritance.

Default Argument:

A default argument is a value in the function declaration automatically assigned by the compiler if the calling function does not pass any value to that argument.

Characteristics for defining the default arguments

Following are the rules of declaring default arguments -

- The values passed in the default arguments are not constant. These values can be overwritten if the value is passed to the function. If not, the previously declared value retains.
- During the calling of function, the values are copied from left to right.
 - All the values that will be given default value will be on the right.

Polymorphism

C++ Polymorphism

The term "Polymorphism" is the combination of "poly" + "morphs" which means many forms. It is a greek word. In object-oriented programming, we use 3 main concepts: inheritance, encapsulation, and polymorphism.

Function Overloading Or Method Overloading

1. Same Name Function
2. With Different parameter.(Also same no parameter but different Data Type)
3. Lies in same class.
4. No inherit Mechanism.

C++ Function Overloading

Function Overloading is defined as the process of having two or more function with the same name, but different in parameters is known as function overloading in C++. In function overloading, the function is redefined by using either different types of arguments or a different number of arguments. It is only through these differences compiler can differentiate between the functions.

The **advantage** of Function overloading is that it increases the readability of the program because you don't need to use different names for the same action.

Constructor in C++

1. Constructor is special member function.
2. Name of Constructor is same as class Name.
3. Constructor will be called automatically, whenever object of the class is created.
4. Constructor does not return any value, not even void.
5. Initial work of Constructor is to initialize object.
6. Constructor can be overloaded.
7. Syntax:<class_name>()

```
{
```

```
Code
```

```
}
```

Example:

Class Demo

```
{
```

```
Public:Demo();
```

```
{}
```

Types of Constructor

1. Default Constructor or No argument or No parameter
2. Parameterized Constructor.
3. Copy Constructor.

Default Constructor: A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

Parameterized Constructor:

A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

C++ Copy Constructor:

A Copy constructor is an **overloaded** constructor used to declare and initialize an object from another object.

Copy Constructor is of two types:

- **Default Copy constructor:** The compiler defines the default copy constructor. If the user defines no copy constructor, compiler supplies its constructor.
- **User Defined constructor:** The programmer defines the user-defined constructor. class as
- Copy constructor takes a reference to an object of the same as an argument.

Destructor in c++

A destructor works opposite to constructor; it destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

A destructor is defined like constructor. It must have same name as class. But it is prefixed with a tilde sign (~).

```
1. #include <iostream>
2. using namespace std;
3. class Employee
4. {
5.     public:
6.         Employee()
7.         {
8.             cout<<"Constructor Invoked"<<endl;
9.         }
10.        ~Employee()
11.        {
12.            cout<<"Destructor Invoked"<<endl;
13.        }
14. };
15. int main(void)
16. {
17.     Employee e1; //creating an object of Employee
18.     Employee e2; //creating an object of Employee
19.     return 0;
```

Static member Function

Static members of a class are not associated with the objects of the class. Just like a static variable once declared is allocated with memory that can't be changed every object points to the same memory.

```
// C++ Program to demonstrate

// Static member in a class

#include <iostream>

using namespace std;

class Student {

public:

    // static member

    static int total;

    // Constructor called

    Student() { total += 1; }

};

int Student::total = 0;

int main()

{

    // Student 1 declared

    Student s1;

    cout << "Number of students:" << s1.total << endl;
```

```

// Student 2 declared

Student s2;

cout << "Number of students:" << s2.total << endl;

// Student 3 declared

Student s3;

cout << "Number of students:" << s3.total << endl;

    return 0;

}

```

Static data member

Static data members are class members that are declared using. Static keywords. A static member has certain special characteristics which are as follows:

- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is initialized by assigning zero before any object of this class is created, even before the main starts.
- It is visible only within the class, but its lifetime is the entire program.

Syntax:

```
static data_type data_member_name;
```

Below is the C++ program to demonstrate the working of static data members

```

// C++ Program to demonstrate
// the working of static data member
#include <iostream>
using namespace std;
class A {
public:
A()
{
    cout << "A's Constructor Called " <<
        endl;
}
}

```

```

    }
};
class B {
    static A a;
public:
    B()
    {
        cout << "B's Constructor Called " <<
            endl;
    }
};
// Driver code
int main()
{
    B b;
    return 0;
}

```

Friend Function

A **friend Function** can access private and protected members of classes in which it is declared as a friend. It is sometimes useful to allow a particular class to access private and protected members of other classes. We can declare a friend function in C++ by using the **friend** keyword.

Syntax:

friend ret type func_name(); // declaration of friend function

or

1. Friend Function is non-member function of class.
2. Friend can access private or protected Data member of class.
3. If you want make any function as friend function ,then in that case we need write friend keyword followed by function name .
4. We Define the signature of friend function inside class public section, whereas definition part outside class.
5. We create n number of friend function in class.
6. We do not call friend function by class name
7. More than one class have the same friend function
8. We need to pass class object as a argument inside friend function

Example:

```

// C++ Program to demonstrate the
// functioning of a friend class
#include <iostream>
using namespace std;

```

```

class GFG {
private:
    int private_variable;

protected:
    int protected_variable;

public:
    GFG()
    {
        private_variable = 10;
        protected_variable = 99;
    }

    // friend class declaration
    friend class F;
};

// Here, class F is declared as a
// friend inside class GFG. Therefore,
// F is a friend of class GFG. Class F
// can access the private members of
// class GFG.
class F {
public:
    void display(GFG& t)
    {
        cout << "The value of Private Variable = "
              << t.private_variable << endl;
        cout << "The value of Protected Variable = "
              << t.protected_variable;
    }
};

// Driver code
int main()
{
    GFG g;
    F fri;
    fri.display(g);
    return 0;
}

```

Output

The value of Private Variable = 10

The value of Protected Variable = 99

- C++

Features of Friend Functions

- A friend function is a special function in C++ that in spite of not being a member function of a class has the privilege to **access the private and protected data** of a class.
- A friend function is a non-member function or ordinary function of a class, which is declared as a friend using the keyword “**friend**” inside the class. By declaring a function as a friend, all the access permissions are given to the function.
- The keyword “friend” is placed only in the function declaration of the friend function and **not** in the **function definition or call**.
- A friend function is called like an ordinary function. It cannot be called using the object name and dot operator. However, it may accept the object as an argument whose value it wants to access.
- A friend function can be declared in any section of the class i.e. public or private or protected.

Below are some more examples of friend functions in different scenarios:

A Function Friendly to Multiple Classes

```
// C++ Program to demonstrate
// how friend functions work as
// a bridge between the classes

#include <iostream>

using namespace std;
// Forward declaration
class ABC;
class XYZ {
    int x;
public:
    void set_data(int a)
    {
        x = a;
    }
    friend void max(XYZ, ABC);
};
class ABC {
    int y;
public:
```

```

void set_data(int a)
{
    y = a;
}
friend void max(XYZ, ABC);
};
void max(XYZ t1, ABC t2)
{

    if (t1.x > t2.y)

        cout << t1.x;

    else

        cout << t2.y;

}

// Driver code

int main()

{

    ABC _abc;

    XYZ _xyz;

    _xyz.set_data(20);

    _abc.set_data(35);

    // calling friend function

    max(_xyz, _abc);

    return 0;

}

```

Output

The friend function provides us with a way to access private data but it also has its demerits. Following is the list of advantages and disadvantages of friend functions in C++:

Advantages of Friend Functions

- A friend function is able to access members without the need of inheriting the class.
- The friend function acts as a bridge between two classes by accessing their private data.
- It can be used to increase the versatility of overloaded operators.
- It can be declared either in the public or private or protected part of the class.

Disadvantages of Friend Functions

- Friend functions have access to private members of a class from outside the class which violates the law of data hiding.
- Friend functions cannot do any run-time polymorphism in their members.

Important Points About Friend Functions and Classes

1. Friends should be used only for limited purposes. Too many functions or external classes are declared as friends of a class with protected or private data access lessens the value of encapsulation of separate classes in object-oriented programming.
2. Friendship is **not mutual**. If class A is a friend of B, then B doesn't become a friend of A automatically.
3. Friendship is not inherited. (See [this](#) for more details)
4. The concept of friends is not in Java.

Operator Overloading

C++ Operators Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type. Operator overloading is used to overload or redefines most of the operators available in C++. It is used to perform the operation on the user-defined data type. For example, C++ provides the ability to add the variables of the user-defined data type that is applied to the built-in data types.

The advantage of Operators overloading is to perform different operations on the same operand.

Operator that cannot be overloaded are as follows:

- Scope operator (::)
- Sizeof
- member selector(.)
- member pointer selector(*)
- ternary operator(?:)

Syntax of Operator Overloading

return_type class_name :: operator op(argument_list)

```
{  
    // body of the function.  
}
```

1. Overloading unary operator.
2. Overloading binary operator.
3. Overloading binary operator using a friend function.

Criteria/Rules to Define the Operator Function

1. In the case of a **non-static member function**, the binary operator should have only one argument and the unary should not have an argument.
2. In the case of a **friend function**, the binary operator should have only two arguments and the unary should have only one argument.
3. All the class member objects should be public if operator overloading is implemented.
4. Operators that cannot be overloaded are `.* :: ?:`
5. Operators that cannot be overloaded when declaring that function as friend function are `= () [] ->`.
6. The operator function must be either a non-static (member function) or a friend function.

Access Specifiers in C++

Access specifiers in C++ define how the members of the class can be accessed. C++ has 3 new keywords introduced, namely.

- public
- private
- protected

Specifiers	Within Same Class	In Derived Class	Outside the Class
Private	Yes	No	No
Protected	Yes	Yes	No
Public	Yes	Yes	Yes

Inheritance in c++

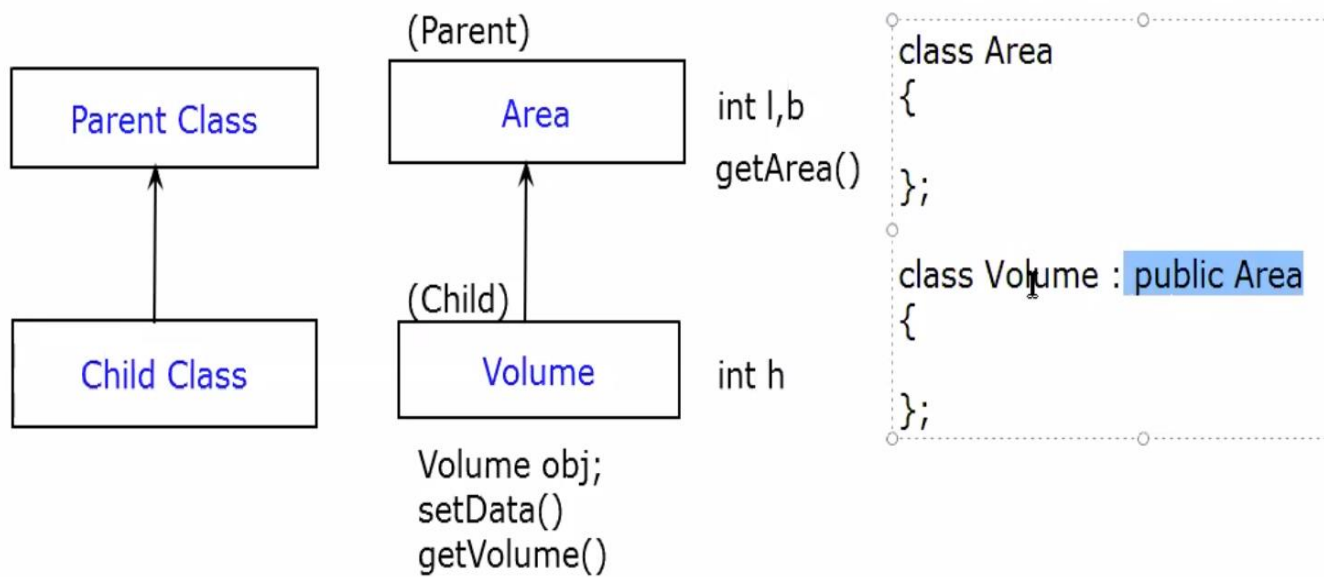
1. Inheritance is the process in which one class inherited the properties of other class.
2. It works on concepts of code reusability.
3. It is hierarchical in nature.
4. It follows parent child relationship, where parent class is known as super class or base and child class is known as sub class or derived class.

Types of Inheritance

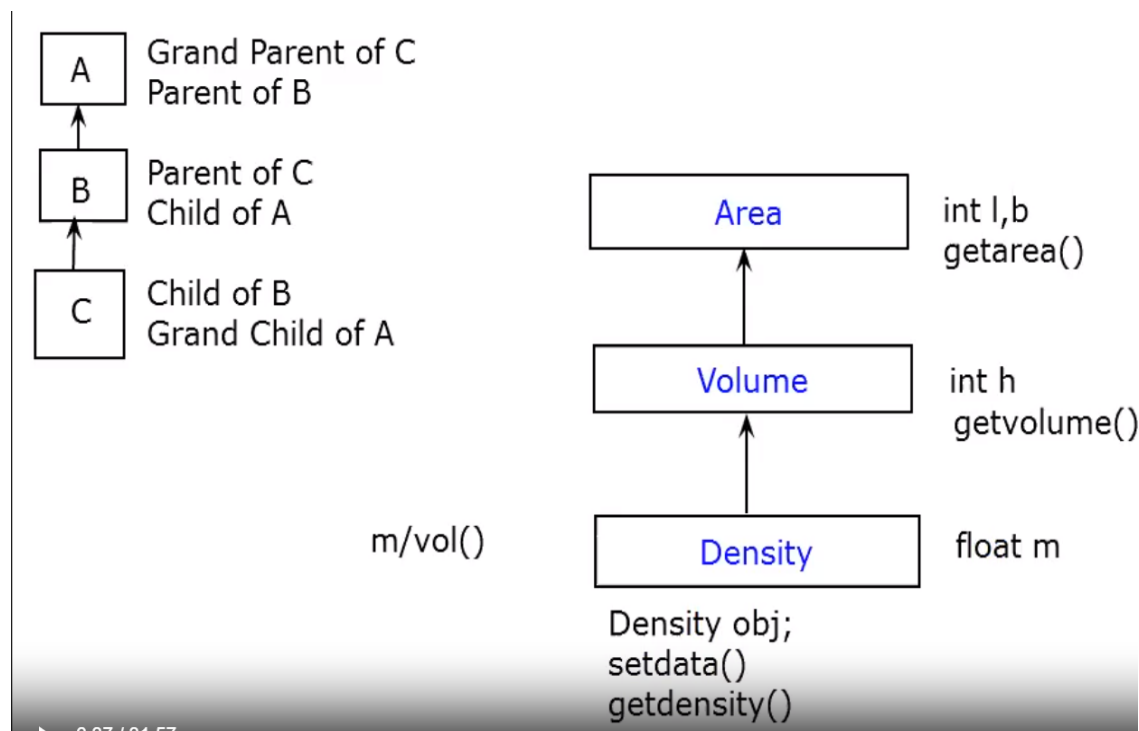
1. Single Inheritance
2. Multilevel Inheritance
3. Multiple Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance.

Single Inheritance:

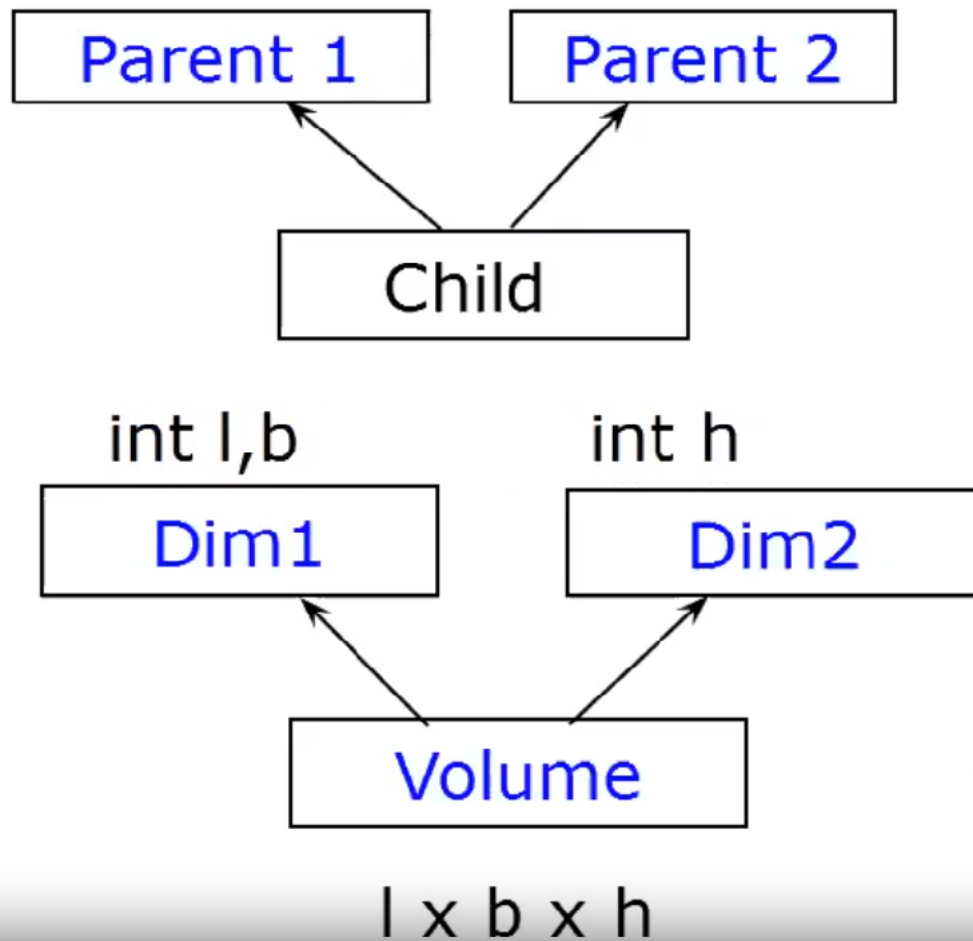
Single Inheritance:



Multilevel Inheritance

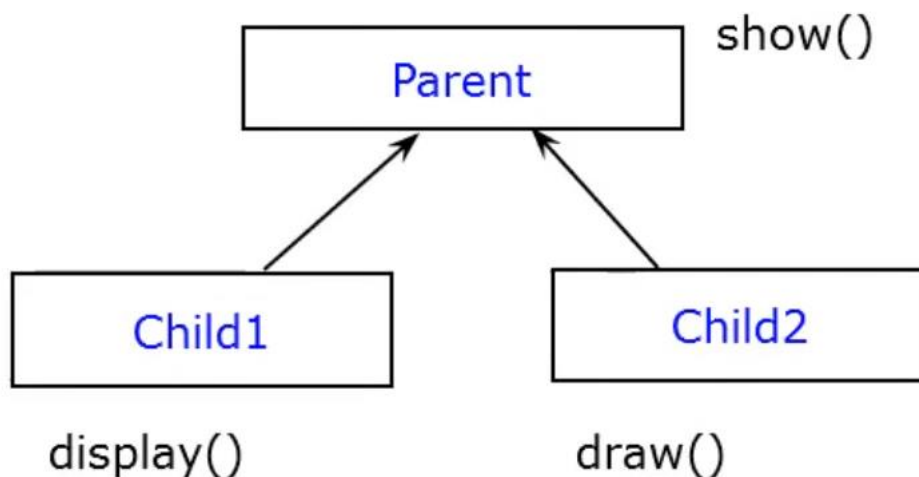


Multiple Inheritances:

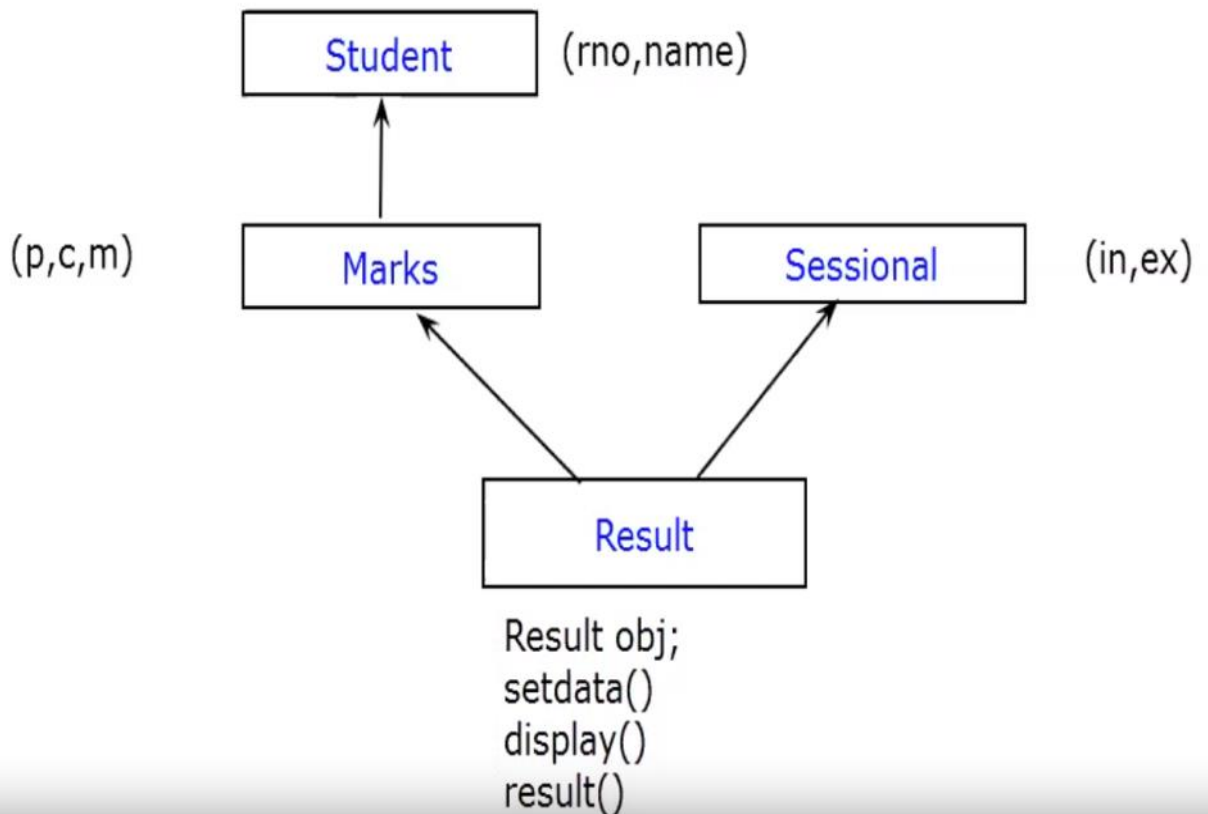


24:51 / 31:57

Hierarchical Inheritance :



Hybrid Inheritance:



What is the Diamond Problem?

When we inherit more than one base class in the same derived class and all these base classes also inherit another but same single class (super parent), multiple references of the super parent class become available to the derived class. So, it becomes unclear to the derived class, which version of the super parent class it should refer to.

```
#include <iostream>  
using namespace std;
```

```
//super parent class  
class A{  
public:  
    void name(){  
        cout << "This is class A \n";  
    }  
};
```

```
//base class I  
class B: public A{};
```

```
//base class II
class C: public A{};

//derived class
class D: public B, public C{};

int main()
{
    D d;
    d.name();

    return 0;
}
```

Solve Dimaond problem :

Virtual inheritance

Virtual inheritance in C++ is a type of inheritance that ensures that only one copy or instance of the base class's members is inherited by the grandchild derived class.

It is implemented by prefixing the `virtual` keyword in the inheritance statement.

C++ Function Overriding

If derived class defines same function as defined in its base class, it is known as function overriding in C++. It is used to achieve runtime polymorphism. It enables you to provide specific implementation of the function which is already provided by its base class.

Pointer to Drive Class and Pointer to base Class:

1. Pointer to Drive class

Example:

```
//cocept pointer to drive class

#include<iostream>

using namespace std;
```

```

class Parent
{
    public:
    void display()
    {
        cout<<"display inside parent "<<endl;
    }
};

class Child : public Parent
{
    public:
    void Show()
    {
        cout<<"show inside child"<<endl;
    }
};

int main()
{
    Child p,*c;
    c=&p;
    c->display();
    c->Show();
    return 0;
}

```

2. Pointer to base class

Example :

```
//concept pointer to Base class
```

```
#include<iostream>
```

```
using namespace std;
```

```
class Parent
```

```
{
```

```
    public:
```

```
    void Display()
```

```
    {
```

```
        cout<<"display inside parent "<<endl;
```

```
    }
```

```
};
```

```
class Child : public Parent
```

```
{
```

```
    public:
```

```
    void Show()
```

```
    {
```

```
        cout<<"show inside child"<<endl;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Parent obj,*bptr;
```

```
    bptr=&obj;
```

```
    bptr->Display();
```

```
((Child*)bptr)->Show();  
  
return 0;  
  
}
```

Virtual Function in c++

Virtual Function:

A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.

- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call.
- They are mainly used to achieve [Runtime polymorphism](#).
- Functions are declared with a **virtual** keyword in a base class.
- The resolving of a function call is done at runtime.

Rules for Virtual Functions

The rules for the virtual functions in C++ are as follows:

1. Virtual functions cannot be static.
2. A virtual function can be a friend function of another class.
3. Virtual functions should be accessed using a pointer or reference of base class type to achieve runtime polymorphism.
4. The prototype of virtual functions should be the same in the base as well as the derived class.
5. They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or re-define the virtual function), in that case, the base class version of the function is used.
6. A class may have a [virtual destructor](#) but it cannot have a virtual constructor.

Pure virtual function and Abstract Class:

A pure virtual function is a virtual function in C++ for which we need not to write any function definition and only we have to declare it. It is declared by assigning 0 in the declaration.

Abstract Class :

- 1) If a class contains pure virtual function, then that class is known as Abstract class. (If we want to make any class as a abstract class, then in that case we need to create pure virtual function in it.)
- 2) Abstract class cannot be instantiated, which means, we cannot create object of abstract class.
- 3) If any class inherits Abstract class, then its mandatory for that class to give the implementation of pure virtual function, otherwise it will give compilation error.
- 4) Abstract class can contains pure virtual function, normal functions and constructor.
- 5) Abstract class provides reference to their child classes.

Exception Handling in C++

Exceptions are runtime anomalies or abnormal conditions that a program encounters during its execution of program, and mechanism of handling he such type of exception known as exception handling.

C++ Exceptions:

When executing C++ code, different errors can occur: coding errors made by the programmer, errors due to wrong input, or other unforeseeable things.

When an error occurs, C++ will normally stop and generate an error message. The technical term for this is: C++ will throw an exception (error).

C++ try and catch:

Exception handling in C++ consists of three keywords: try, throw and catch:

The try statement allows you to define a block of code to be tested for errors while it is being executed.

The throw keyword throws an exception when a problem is detected, which lets us create a custom error.

The catch statement allows you to define a block of code to be executed if an error occurs in the try block

```
try
{
    -----
    detect the exception and throw the exception
    -----
}
catch(.....)
{
    handles the exception
}
```

```
function()
{
    generates exception and throw exception
}

try
{
    -----
    function calling();
    -----
}
catch(.....)
{
    handles the exception
}
```

try with multiple catch

```
try
{
    -----
}
catch(...)
{
    -----
}
catch(...)
{
    -----
}
catch(...)
{
    -----
}
```

try with generalized catch

```
try
{
    -----
}
catch(...)
{
    -----
}
```

Inline Function in c++:

C++ **inline** function is powerful concept that is commonly used with classes. If a function is inline, the compiler places a copy of the code of that function at each point where the function is called at compile time.

To inline a function, place the keyword **inline** before the function name and define the function before any calls are made to the function. The compiler can ignore the inline qualifier in case defined function is more than a line.

A function definition in a class definition is an inline function definition, even without the use of the **inline** specifier.

Following is an example, which makes use of inline function to return max of two numbers –

```
#include <iostream>
using namespace std;
inline int Max(int x, int y) {
    return (x > y)? x : y;
}
// Main function for the program
int main() {
    cout << "Max (20,10): " << Max(20,10) << endl;
```

```
cout << "Max (0,200): " << Max(0,200) << endl;
cout << "Max (100,1010): " << Max(100,1010) << endl;
return 0;
}
```

Dynamic memory allocation in c++

Dynamic memory allocation in C/C++ refers to performing memory allocation manually by a programmer. Dynamically allocated memory is allocated on **Heap**, and non-static and local variables get memory allocated on **Stack**

There two operator are used in c++ memory allocation

1. **New**
2. **delete**

new operator

The new operator denotes a request for memory allocation on the Free Store. If sufficient memory is available, a new operator initializes the memory and returns the address of the newly allocated and initialized memory to the pointer variable.

Syntax to use new operator

pointer-variable = **new** data-type;

deleteoperator:

Since it is the programmer's responsibility to deallocate dynamically allocated memory, programmers are provided delete operator in C++ language.

Syntax:

// Release memory pointed by pointer-variable

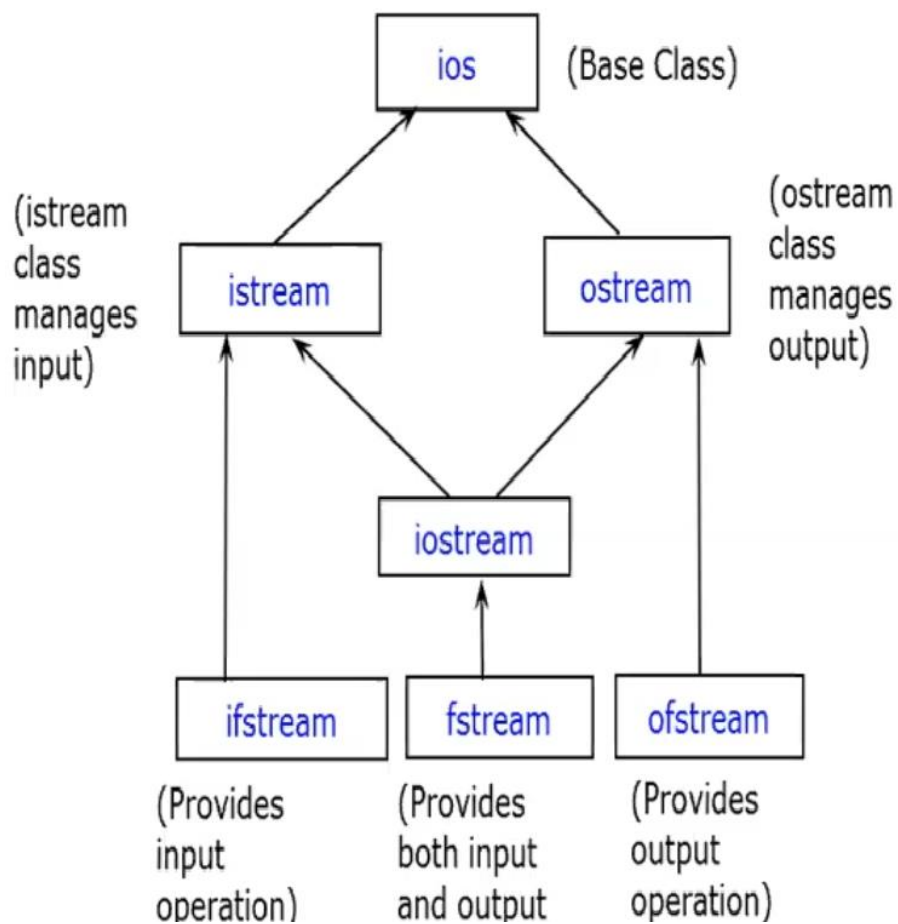
delete pointer-variable;

File handling in c++

File Handling In C++

- 1) Secondary storage devices such as hard disk or floppy disk are used to store large amount of data, and the data is stored on these devices using the concept of files.
- 2) A file is a collection of interrelated records, which comprises of data items, stored in a particular area on the disk.
- 3) Programs can perform read and write operations on these files.
- 4) C++ I/O system uses file streams as an interface between the programs and the files.
- 5) There are two types of streams in the I/O system of C++.
 - i) Input Stream
 - ii) Output Stream

Stream Classes



File Modes

`stream_object.open("file_name","mode");`

Mode	Description
<code>ios::in</code>	Open file for reading only
<code>ios::out</code>	Open file for writing only
<code>ios::app</code>	Append to the end-of-file
<code>ios::trunc</code>	Delete contents of the file if it exists
<code>ios::nocreate</code>	Open fails if the file does not exists
<code>ios::noreplace</code>	Open fails if the file already exists